

Lecture 1: Principles of Secure Software Design

Defence in Depth This involves multiple layers of security controls to protect an organization’s assets. If one layer fails, others still provide protection.

Fail Safe When a system error occurs, the system should safeguard the secrecy, accuracy, and availability of information. The system should revert to a safe condition in the event of a breakdown or malfunction.

Least Privilege Providing individuals or processes with the minimal access rights required to accomplish their tasks.

Separation of Duties A fundamental principle of internal control in business and organizations. It ensures that no single individual has control over all aspects of a transaction by enforcing a system of checks and balances.

Economy of Mechanism Keeping systems simple and small, thereby reducing the chances of security vulnerabilities.

Complete Mediation Every access request must be checked against the access control mechanism.

Open Design The security of a system should not rely on the secrecy of its design. Implementation details should be independent of the design itself, allowing the design to remain open while implementation details may be kept secret.

Least Common Mechanism Minimizing the amount of mechanisms shared by different users or processes in order to reduce the risk of a single breach affecting multiple users.

Psychological Acceptability Security mechanisms should be easy to use and acceptable to users so that security functionality is used correctly and consistently.

Weakest Link The overall security of a system depends heavily on its weakest component, whether that is code, a service, or an interface.

Leveraging Existing Components Reusing existing, well tested software components and functionality to avoid introducing new vulnerabilities and unnecessarily increasing the attack surface.

Lecture 2: Vulnerability vs Threat vs Asset

Definition of Software Vulnerability: A security flaw, glitch, or weakness found in software code that could be exploited by an attacker.

Definition of Threats: Any circumstance or event with the potential to cause harm to a system in the form of destruction, disclosure, modification of data, and/or denial of service.

Threat Agent: This is the means or tool used by the threat actor to execute the attack.

Backdoor: A backdoor is an illicitly created or pre-existing access point in a system that enables an unauthorized party to access the system without going through the

normal authentication and authorization processes.

Phishing: Phishing is a form of social engineering attack. Attackers use deceptive emails and forged websites to carry out online fraud activities.

Man-in-the-middle (MITM) attacks: A man-in-the-middle (MITM) attack is an attack in which a hacker steals your information by getting in between you and a trusted party.

Denial-of-Service (DoS) Attacks: A DoS attack is a malicious attempt to disrupt the normal functioning of a targeted server, service, or network by overwhelming it with a flood of traffic or requests.

Broken Access Control: Access control enforces policy such that users cannot act outside of their intended permissions.

Injection: Injection refers to a security flaw where attackers insert malicious code or commands into software applications, causing them to execute unintended operations.

Secured Software Development Lifecycles (SDLC): Requirements, Analysis, Design, Implementation, Testing, Maintenance.

Lecture 3: Access Control — Authentication & Authorization

Authentication: Authentication is the process of verifying the identity of a user, device, or entity before allowing access to a system.

Authorization: Authorization determines what an authenticated user is allowed to do, specifying access rights and permissions to resources.

Authentication vs Authorization: Authentication answers the question *who you are*, while authorization answers the question *what you are allowed to do*.

Access Control: Access control enforces security policies to ensure that users cannot act outside of their intended permissions.

Credentials: Credentials are pieces of information used to authenticate an identity, such as usernames, passwords, tokens, or keys.

Authentication Mechanisms

Two-Factor Authentication (2FA) / Multi-Factor Authentication (MFA): Authentication mechanisms that require users to provide two or more independent authentication factors to verify their identity.

Access Control Models

Discretionary Access Control (DAC): Resource owners decide on access rights. (Note: DAC employs Access Control Lists (ACLs) for defining permissions.)

Mandatory Access Control (MAC): Access decisions are based on predetermined security labels assigned to users and data.

Role-Based Access Control (RBAC): Access rights are assigned based on the user’s role within the organization. (Note: Roles are assigned based on job competencies, authority, and responsibility levels)

Attribute-Based Access Control (ABAC): An access control method where subject requests to perform operations on objects are granted or denied based on assigned attributes of the subject, assigned attributes of the object, environment conditions, and a set of policies that are specified in terms of those attributes and conditions.

Lecture 4: Threat Modelling

Threat Modelling: Threat modelling is a structured approach used to identify, analyze, and address potential security threats to a system during its design and development.

Threat Modelling in SDLC: Threat modelling is typically performed during the design phase of the SDLC, but can be revisited iteratively as the system evolves.

Threat Actor: A threat actor, also known as a cyber threat actor or malicious actor, is an individual or group that deliberately causes damage to digital devices or systems.

Purpose of Threat Modelling: The goal of threat modelling is to systematically identify potential threats, understand their impact, and support secure design decisions early in the software development lifecycle.

Attack Surface: The attack surface represents the set of entry points, interfaces, and interactions through which an attacker could attempt to compromise a system.

Attack Tree: An attack tree is a structured, hierarchical representation of the different ways an attacker can achieve a specific malicious goal against a system.

Attack-Defence Tree: An attack-defence tree extends an attack tree by incorporating defensive measures that can be applied to counter or mitigate specific attack paths.

Threat Libraries: Threat libraries (e.g, CWE, CAPEC) are collections of known threats and attack patterns that can be reused during threat modelling to help identify potential security threats.

STRIDE Threat Classification Model

STRIDE: STRIDE is a threat classification model used to identify different categories of security threats.

- Spoofing: Spoofing threats involve an adversary creating and exploiting confusion about who is talking to whom.
- Tampering: This refers to the unauthorized alteration of data. Tampering is any activity that causes illegal alterations or data exploitation.
- Repudiation: This threat involves performing an illegal operation in a system and then denying having performed that operation.
- Information Disclosure: Exposing information to

someone not authorized to see it.

- Denial of Service: (see Lecture 2)
- Elevation of Privilege: Elevation of privilege threat refers to an attacker obtaining access privileges higher than their legitimate rights by exploiting vulnerabilities or errors.

Four Types of Actions Against a Threat: (1) Avoid the threat; (2)Mitigate the threat; (3)Transfer the threat; (4) Accept the threat.

Lecture 5: Misuse Cases

Misuse Case (Def.): A misuse case is essentially an “inside-out” use case, focusing on features that a system should not allow. It is an attempt to elicit security requirements by considering what a malicious actor could do within the context of the system. A misuse case is always associated with a misactor (misuser), who may deliberately or accidentally cause damage to the system.

New Relations in Misuse Case Diagrams: In addition, it introduces two new relations to be used in the diagram: «Mitigates» A use case can mitigate the chance that a misuse case will complete successfully. «Threatens»: A misuse case can threaten a use case, e.g. by exploiting it or hinder it from achieving its goals.

Lecture 6: Security Patterns

Security Pattern: Security patterns are reusable solutions to common security problems that occur in software design and implementation. They provide proven approaches for addressing recurring security concerns in a structured and systematic way.

Common Security Patterns

Authentication & Authorization: Security patterns that ensure only authenticated users can access a system and that authenticated users are permitted to perform only authorized actions.

Data Access Object (DAO): A design pattern that provides an abstract interface to a database or other persistence mechanism, separating data access logic from business logic and helping enforce controlled access to data.

Input Validation: A security pattern that ensures all input data is checked for correctness, completeness, and validity before being processed, in order to prevent malicious or malformed input from causing security vulnerabilities.

Output Encoding / Escaping: A security pattern used to prevent injection attacks by transforming special characters into a safe representation before output is rendered or interpreted by another system.

Secure Error Handling Patterns: Security patterns that focus on managing errors and exceptions without exposing sensitive information or internal system details to users or attackers.

Secure Logger: A security pattern that ensures logging mechanisms capture relevant security events while avoiding the disclosure of sensitive information and protecting log integrity.

Secure Communication: A security pattern that protects data exchanged between systems or components by ensuring confidentiality, integrity, and authentication during transmission.

Data Sanitization: A security pattern that involves deliberately and permanently deleting or destroying data so that it cannot be recovered, ensuring sensitive information is not exposed after its lifecycle ends.

Secure Default: A security pattern where the default configuration of a system enforces restrictive and conservative security policies, following a “deny unless explicitly authorized” approach.

Least Privilege: A security pattern that restricts users, processes, and systems to the minimum level of access rights necessary to perform their intended functions.

Lecture 7: Static Analysis & Refactoring

Static Analysis: Static analysis examines a program without executing it, by analysing the source code or byte-code to identify potential errors, bugs, or security vulnerabilities.

Control Flow Analysis: Control flow analysis examines the order in which individual statements, instructions, or function calls are executed in a program, and is used to reason about possible execution paths and control structures.

Data Flow Analysis: Data flow analysis tracks how data values propagate through variables, expressions, and functions in a program, and is used to detect issues such as improper data usage or missing validation.

Taint Analysis: Taint analysis is a form of data flow analysis that tracks the flow of untrusted or tainted input through a program to determine whether it reaches sensitive operations without proper validation or sanitisation.

Taint Sources: Taint sources are inputs that originate from untrusted or external entities, such as user input, network input, files, or environment variables.

Taint Propagation: Taint propagation describes how tainted data flows through variables, expressions, function calls, and program paths as the program executes.

Sensitive Sinks: Sensitive sinks are security-critical operations where the use of tainted data may lead to vulnerabilities, such as database queries, command execution, file access, or output rendering.

Refactoring: Refactoring is the process of improving the internal structure of code without changing its external behaviour, with the goal of improving readability, maintainability, and security.

Lecture 8: Confidentiality & Integrity in

Software

CIA Triad: The CIA Triad is a model guiding information security policies within organizations, encompassing Confidentiality, Integrity, and Availability.

Confidentiality: Information and functions can be accessed only by properly authorized parties.

Integrity: Information and functions can be added, altered, or removed only by authorized persons and means, preventing improper or unauthorized change.

Confidentiality Policy (Information Flow Policy): Measures to ensure that information is accessible only to those authorized to have access. A confidentiality policy is also called an information flow policy.

Information Flow Control: Concerned with how information is disseminated or propagated from one object to another. Information flow policies prevent unauthorized disclosure of information.

Bell–LaPadula (BLP) Model

Bell–LaPadula Model: An information confidentiality model that combines mandatory security levels with discretionary access control.

System Entities: The system consists of subjects and objects.

Security Levels: Each subject and object is assigned a security level.

Simple Security Property (ss-property): No read up. A subject *s* may read an object *o* only if $L(s) \geq L(o)$.

Star Security Property (*-property): No write down. A subject *s* may write to an object *o* only if $L(s) \leq L(o)$ and *s* has permission to write *o*.

Discretionary Security Property: Access is also constrained by an access matrix specifying permitted operations.

BLP Concept: Write up, read down (WURD).

Integrity Policies

Integrity: Integrity refers to the trustworthiness of data or resources and is defined in terms of preventing improper or unauthorized modification.

Integrity Levels: Integrity levels are labels that specify the degree of confidence placed in data or subjects.

Biba Integrity Model

Biba Integrity Model: An integrity model designed to prevent data from being corrupted by untrusted or low-integrity subjects.

Model Components: A system consists of a set of subjects *S*, a set of objects *O*, and a set of integrity levels *I*. A function $i : S \cup O \rightarrow I$ assigns integrity levels.

Simple Integrity Axiom: No read down. A subject *s* may read an object *o* only if $i(s) \leq i(o)$.

Star Integrity Axiom: No write up. A subject *s* may write to an object *o* only if $i(o) \leq i(s)$.

Invocation Property: A subject *s*₁ may invoke or execute another subject *s*₂ only if $i(s_2) \leq i(s_1)$.

Biba Low Watermark Integrity Policy

Low Watermark Policy: A subject’s integrity level is dynamically reduced to the lowest integrity level of any object it has read.

Low Watermark Rule: If subject *s* reads object *o*, then the new integrity level of *s* becomes $i'(s) = \min(i(s), i(o))$.

Write Rule under Low Watermark: A subject *s* may write to object *o* only if $i(o) \leq i(s)$.

Invocation Rule: The invocation property remains unchanged.

Lecture 9: Secure Coding & Security Testing

Secure Coding (Definition): Secure coding enables organisations to prevent security risks and vulnerabilities that may arise as a result of poor software coding practices.

Security Testing: Security testing evaluates a system to identify vulnerabilities, weaknesses, and risks that could be exploited by attackers.

Common Testing / Verification Approaches

Fuzzing (Testing): A testing technique that feeds invalid, unexpected, or random data as inputs to a program in order to identify crashes, unexpected behaviour, or security vulnerabilities.

Penetration Testing: A security testing approach that simulates real-world attacks to evaluate the security of a system.

Run-time Verification: A verification approach that monitors the execution of a system at run time to detect violations of specified security or correctness properties.

Lecture 10: SSE Applications and Regulatory

Biometrics (Definition): Biometrics are biological measurements or physical characteristics that can be used to identify individuals.

Fingerprint Recognition: A biometric system where a scanner captures a digital representation of a user’s fingerprint, extracts features, stores fingerprint templates, and compares scanned fingerprints with stored templates for verification or identification.

Face Recognition: A biometric technology used to identify or confirm an individual’s identity using their face.

Iris Recognition: A biometric technique that recognises a person by analysing the random pattern of the iris.

Voice Recognition: A biometric technology that identifies a person’s voice and can interpret speech.

GDPR (Definition): The General Data Protection Regulation (GDPR) is a regulation in EU law on data protection and privacy in the European Union and the European

Economic Area.

Key Principles of GDPR (Article 5)

- Lawfulness, Fairness, and Transparency
- Purpose Limitation
- Data Minimisation
- Accuracy
- Storage Limitation
- Integrity and Confidentiality

Rights of Data Subjects (Chapter 3)

Right to Access (Art. 15): The data subject has the right to obtain confirmation as to whether personal data are being processed and access to those data.

Right to Rectification (Art. 16): The data subject has the right to obtain rectification of inaccurate personal data.

Right to Erasure / Right to be Forgotten (Art. 17): The data subject has the right to obtain the erasure of personal data without undue delay.

Right to Restrict Processing (Art. 18): The data subject has the right to restrict the processing of personal data.

Right to Data Portability (Art. 20): The data subject has the right to receive personal data in a structured, commonly used, and machine-readable format and transmit it to another controller.

Right to Object (Art. 21): The data subject has the right to object to processing of personal data, including for direct marketing purposes.

Impact of GDPR on Software Development and Data Handling

Consent (Art. 7, 8): Consent must be explicit, demonstrable, and withdrawable. Special rules apply to children’s data.

Personal Data Breaches (Art. 34): When a personal data breach is likely to result in high risk to individuals’ rights and freedoms, the data subject must be informed without undue delay.

Privacy by Design and by Default (Art. 25): Appropriate technical and organisational measures must be implemented to ensure that only necessary personal data are processed.

Data Protection Impact Assessment (DPIA) (Art. 35): DPIAs are required where data processing is likely to result in high risk to individuals’ rights and freedoms.

Data Transfers (Art. 44–49): Personal data transfers outside the EU/EEA are permitted only under specified safeguards and conditions.

Data Protection Officer (DPO) (Art. 37–39): Certain organisations must appoint a DPO, defining their responsibilities and independence.